BrainAI

ProSuite Industrial

CORE ARCHITECTURE

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**ProED Engine Manual**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2025-01-01	S. Köhne	Stable	Initial Core Engine
2.0.0	2026-02-26	S. Köhne	Stable	Hybrid Ring-0 Edition (ProKM Kernel Support & Armored Envelope)

1. Product Overview

The ProED Core Engine v1.1.0 is an extremely high-performance, hybrid cryptography module for high-security environments. It provides branchless encryption and is designed to process data both in user-space (Ring-3) and hardware-accelerated directly within the operating system kernel (Ring-0 via ProKM).

Core Features

- **Hybrid Architecture:** Automatic detection and utilization of the `ProKM` kernel driver for maximum I/O performance. If the system falls back to an environment without the driver, the native shared library (DLL/SO) seamlessly takes over.
- **Armored Envelope (AEAD):** The software implementation automatically encapsulates the payload in a cryptographic vault ("Envelope"). This includes the Initialization Vector (IV) and a constant-time HMAC to prevent manipulation before decryption.
- **Zero-Allocation in Kernel:** In Ring-0 mode, the engine operates completely memory-efficient without dynamic memory allocations (zero-copy approaches are possible).
- **Side-Channel Immunity:** All core algorithms are implemented to be memory- and time-constant (constant-time) to physically exclude cache and timing attacks.

2. Technical Specifications

2.1 Global Constants

These values are mandatory for all implementations and ensure memory safety during allocation:

- **HEADER_SIZE:** *48* Bytes (Fixed size of the "Armored Envelope" header).
- **KEY_SIZE:** Exactly *32* Bytes (*256*-bit entropy).
- **CTX_SIZE:** *120* Bytes (Size of the internal state context in RAM).

2.2 Packet Layout (Software Armored Envelope)

When data is processed in user-space (`EncryptEnvelopeSoftware`), ProED generates a continuous binary block with the following structure:

- **Byte 00 – 15:** Initialization Vector (IV) in plaintext.
- **Byte 16 – 47:** Constant-Time Auth-Tag (HMAC) for integrity verification.
- **Byte 48 – End:** Encrypted payload.

Note: In Kernel-Mode (`ProcessKernel`), metadata and IV are passed via an IOCTL struct, and only the pure payload is returned.

3. Global Error Management

The engine utilizes a direct return-value model in its C-interface, which is evaluated by the wrappers.

Code	***Meaning***	***Description***	***Criticality***
1	SUCCESS	Operation completed successfully. Payload is authentic.	*None*
0	FAILURE	Operation aborted. Either a parameter error or (during decrypt) the HMAC check failed. The packet was manipulated!	*High (Attack!)*

4. C# (.NET) Integration & Hybrid Handling

The *C# Wrapper* abstracts the entire system complexity. Upon initialization, it uses `CreateFile` to check if the Windows kernel driver (`\\.\ProKM`) is accessible and routes the data intelligently.

4.1 Implementation Example

```
using ProED;

public void HybridEncryptionExample()
{
    byte[] sessionKey = GetSecureKey32Bytes();
    byte[] iv = GetSecureRandom16Bytes();
    byte[] secretData = System.Text.Encoding.UTF8.GetBytes("Critical

    // Using block ensures kernel handles are released (if open)
    using (var proed = new ProEDClient(sessionKey))
    {
        byte[] encrypted;

        if (proed.IsKernelModeActive)
        {
            // Maximum performance: Processing directly in Ring-0
            LogInfo("ProKM Kernel Driver found! Routing data through
            encrypted = proed.ProcessKernel(secretData, iv, isDecrypt
        }
        else
        {
            // Maximum compatibility: Processing in user-space with A
            LogInfo("No driver found. Using C-DLL fallback (Armored E
            encrypted = proed.EncryptEnvelopeSoftware(secretData);
        }

        if (encrypted == null)
        {
            throw new SecurityException("Encryption failed at the eng
        }
```

```
    }
}
```

5. Python (High-Performance Scripting)

The *Python Wrapper* combines `ctypes` with the `struct` module to send native `DeviceIoControl` calls to the driver on Windows systems, while seamlessly utilizing the shared library (`.so`) on Linux.

5.1 Implementation Example

```
from proed import ProED

def process_sensor_data(sensor_payload: bytes, key: bytes, iv: bytes)
    # Context manager (__enter__ / __exit__) ensures handle releases
    with ProED(key) as engine:

        try:
            if engine.is_kernel_mode_active:
                print("Hardware acceleration active.")
                encrypted = engine.process_kernel(sensor_payload, iv,
            else:
                print("Software mode active.")
                encrypted = engine.encrypt_envelope_software(sensor_p

            if encrypted is None:
                raise ValueError("Cryptographic error.")

            transmit_to_cloud(encrypted)

        except OSError as e:
            print(f"I/O Error during kernel communication: {e}")
```

6. Java (JNA / Enterprise Integration)

The *Java* implementation leverages JNA (Java Native Access) to break through the JVM's system boundaries. It loads both the ProED library and `kernel32.dll` to control kernel memory access directly from the Java backend.

6.1 Implementation Example

```
import com.brainai.proed.ProED;

public class SecurityService {
    public byte[] decryptIncoming(byte[] networkPacket, byte[] sessio
        // Try-With-Resources for IDisposable pattern
        try (ProED engine = new ProED(sessionKey)) {

            if (engine.isKernelModeActive()) {
                // Assumes the IV was transmitted separately via the
                byte[] iv = extractIvFromProtocol(networkPacket);
                byte[] rawCipher = extractCipherFromProtocol(networkP
                return engine.processKernel(rawCipher, iv, true);
            } else {
                // Decrypts the full 48-byte envelope including MAC
                byte[] decrypted = engine.decryptEnvelopeSoftware(net

                if (decrypted == null) {
                    System.err.println("CRITICAL: MAC Error. Packet w
                    triggerSecurityAlert();
                }
                return decrypted;
            }
        }
    }
}
```

7. JavaScript (Node.js / Web-Services)

The *Node.js* wrapper utilizes `ffi-napi` and native Buffers. It is designed to pass massive asynchronous I/O streams directly to the driver in an extremely memory-efficient manner (zero-copy).

7.1 Implementation Example

```javascript
const ProED = require('./proed');

function secureDataStream(rawData, keyBuffer, ivBuffer) {
    const engine = new ProED(keyBuffer);
    let cipherBuffer = null;

    if (engine.isKernelModeActive) {
        console.log("Using Ring-0 for high throughput.");
        cipherBuffer = engine.processKernel(rawData, ivBuffer, false)
    } else {
        console.log("Using Ring-3 fallback.");
        cipherBuffer = engine.encryptEnvelopeSoftware(rawData);
    }

    // IMPORTANT: Explicitly release the kernel memory handle
    engine.close();

    if (!cipherBuffer) {
        throw new Error("Encryption failed.");
    }
    return cipherBuffer;
}
```

8. VB.NET (Industrial Systems)

In industrial automation (OT / PLC controls), the VB.NET wrapper secures facilities against manipulation. The `IDisposable` interface guarantees the stability of long-running systems (preventing handle leaks).

8.1 Implementation Example

```
Public Sub HandleMachineCommand(packet As Byte(), key As Byte())
    ' Using block guarantees the CloseHandle() call even if exception
    Using engine As New ProEDClient(key)

        Dim plainCommand As Byte()

        If engine.IsKernelModeActive Then
            Dim iv As Byte() = ExtractIv(packet)
            Dim cipher As Byte() = ExtractCipher(packet)
            plainCommand = engine.ProcessKernel(cipher, iv, True)
        Else
            plainCommand = engine.DecryptEnvelopeSoftware(packet)
        End If

        If plainCommand Is Nothing Then
            RaiseEvent SecurityAlert("Integrity violation! Emergency
            MachineControl.EmergencyStop()
            Exit Sub
        End If

        MachineControl.ExecuteCommand(plainCommand)
    End Using
End Sub
```

9. Best Practices & Security Guidelines

- **Resource Management (Kernel Handles):** The hybrid wrapper accesses the `ProKM` driver at the system level (Ring-0). All instances **must** be closed after use (`Dispose`, `close()`, `with`-statement) to prevent memory and handle leaks within the operating system.
- **Nonce/IV Management:** An Initialization Vector (IV) must *never* be reused under the same key. Source your IVs exclusively from true hardware entropy sources (such as the `ProKey` engine).
- **Buffer Wiping:** The C-engine automatically and securely wipes its internal states (`ProED_Context`) from RAM. However, in high-level languages (C#, Java), it is up to the Garbage Collector to free memory. Highly sensitive plaintext buffers should be manually overwritten with zeros after use.
- **Driver Deployment:** The `ProKM` driver must be installed with administrator privileges and properly signed. If these privileges are missing at runtime, the wrapper silently and stably falls back to the "Armored Envelope" software encryption.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**